Robust Fiducial Markers Detection And Following Using Quadruped Robot

Group #1, Construction Robotics, WS 2022/23

Utku Elagoz (532909), Moayed Mohamed Hani Ibrahim Elmoursi (535322), Rami Nasr (538550), Ararat Cetinkaya (535364), and Abdelrahman Yousef Elhossiny Abounida (532925)

***Abstract*—Navigation is one of the most challenging competencies required for mobile robots, especially in industrial applications. When it comes to hazardous or closed environments, existing navigation approaches use high-level sensors which are cost-demanding, such as GPS (which is not functional in specific indoor environments with no signal) and high-level cameras. Therefore, a computationally efficient approach using low-cost cameras and robots is required. This paper presents a novel method for robust detection and following of fiducial markers using quadruped robots. The method involves the use of computer vision techniques to detect markers in real-time and localize the robot, a control algorithm to enable the robot to follow a dynamically moving marker smoothly and accurately, and a Convolutional Neural Network model for marker detection to improve the robustness and continuity of the system. The complete Apriltag detection process was evaluated in both a real-world environment and in Gazebo simulation. The results demonstrated that the proposed method is highly robust and effective in achieving stable marker tracking. This work has the potential to enable a wide new range of applications of quadruped robots in areas such as industrial inspection and construction sites.**

***Index Terms*—Petoi-Dog, AprilTags, Localization, Moving Average Filter, Servo control, CNN-Convenutional Neural Network**

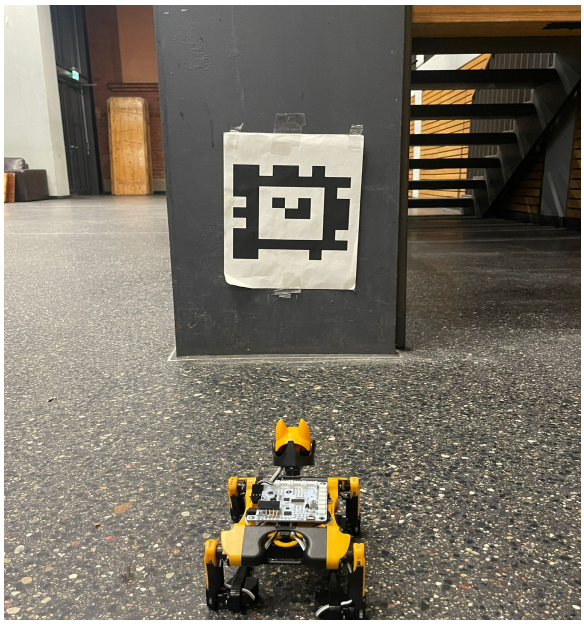


Fig. 1. Quardruped Robot setup

I. Introduction

Navigation and path following require both localization and control to be realized. Localization is the process of computing the position of an object with respect to a coordinate frame. Such algorithms use available information about an object's position and state to determine its position relative to a specified coordinate frame. In this paper, AprilTags are used to localize a Petoi-Dog robot. The robot is a low-cost, open source programmable quadruped robot. The hardware consists of a Raspberry Pi 3A+ and a NyBoard V1, along with a camera that's used for tag detection (see Fig.1).

This paper will cover the aspects of camera calibration, detection of the fiducial markers and localizing the Petoi-Dog based on these detections, a proportional control algorithm to allow tracking of a dynamically moving tag and a Conventional Neural Network model for marker detection to improve the robustness, speed, and accuracy of the system. Often, localization measurements from such tags are noisy and must be filtered. Therefore, a moving average filter is used. Assuming the motion isn't excessive between frames, filtering would aid in smoothing the noise from our readings. The motion of the robot can be controlled by the servo motors in the arms and shoulders which are actuated based on the error between the target and the initial position. Finally, the localization accuracy and tag detection are tested in a real environment, further merged with the control algorithm to verify the effectiveness of this method in dynamic tracking, and lastly enhanced with the CNN model.

II. Scientific Review

Path following is a crucial aspect of autonomous robotics, which requires the development of algorithms and systems that enable robots to move along predetermined paths. The use of AprilTags, which are fiducial markers, in path following has been significantly researched in recent years. The robustness, scalability, and low computational requirements of these tags make them well-suited for use in real-world robotic applications.

AprilTags have many real-world applications with path-following robots and are applied in various domains, such as agriculture, industrial automation, environmental monitoring

and conducting undertakings in hazardous environments. In agriculture, they are used for tasks such as mapping and monitoring crops, and spraying pesticides [1].

Path-following robots using AprilTags are also used in environmental monitoring to monitor and map environments, such as forests, for research purposes and to monitor changes in the environment over time [2]. Furthermore, in industrial automation, they are used to guide robots in assembly lines and material handling [3].

AprilTags have several advantages that make them well-suited for path-following applications. They are robust, have low detection error rates, and are highly scalable making them reliable for use in real-world environments and large environments with large numbers of robots [4] [5]. Additionally, such tags aren't computationally demanding, making them suitable for use in systems with limited computational resources [6].

Despite the many advantages of AprilTags, there are also several challenges associated with their use in path-following applications. The first major challenge is the appearance of the tags in different lighting conditions, which can lead to difficulties in detecting and tracking the markers [7]. Another challenge is the difficulty in detecting markers that are far away or in peripheral view, resulting in a limited range of AprilTags detection [8].

III. Methodology

Detecting AprilTags and estimating camera position precisely, requires a well-processed image. In this paper, a complete navigation system will be implemented starting with modeling camera intrinsic and extrinsic Parameters (Camera Calibration), detecting AprilTag in the processed image, estimating the robot 2D position (AprilTag Detection) and navigating towards the detected AprilTag (Controller).

A. Camera Calibration

Basically, a camera converts light particles that hit the camera's lens image sensor to digital images consisting of pixels, which are the smallest units of digital images.

There are various camera options used in robotics. Some examples are RGB cameras and 3D cameras, nevertheless, none of those cameras are perfect to transform the real world into digital images [9].

Explaining camera calibration requires a camera model which explains the relationship between a point in the world and the projection on the image plane. The most basic one is the camera pinhole model.

1) Pinhole Camera Model: A pinhole camera is a simple camera without a lens and with a single small aperture. Light rays pass through the aperture and project an inverted image on the opposite side of the camera, mirroring the image plane at the focal point creates an upright virtual image of the figure.2

The pinhole camera parameters are represented in a 3-by-4 matrix called the camera matrix. This matrix maps the 3-D world scene into the image plane.

Those parameters are the focal length which is the distance between the pinhole and the image plane, the camera center

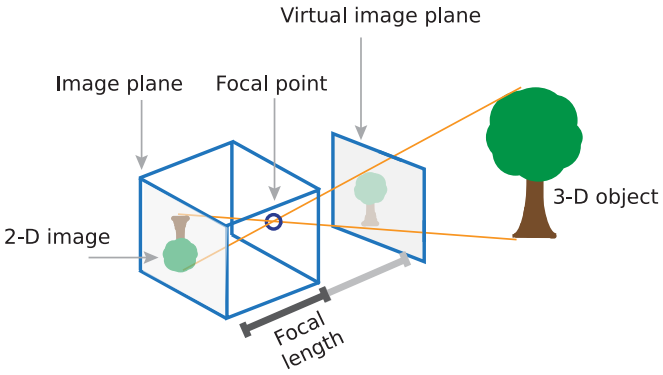


Fig. 2. Pin hole camera model, [21]

which represents the coordinates of the center of the pinhole, beside the other parameters, which will be discussed in more detail later.

Now, the camera can be located anywhere in the world and it can be oriented in any way. So, one has to figure out how it projects the points from the world space onto the image plane. In other words, the relationship between points in the world and their corresponding pixels on the image captured by the camera.

Consequently, calibrating the camera requires 2 transformations. The first one is responsible for changing the basis transformation from the world coordinate system to the camera coordinate system, which is called camera pose or camera extrinsic. The latter one projects the points in the world viewed from the camera onto the image plane of the camera, which is called camera intrinsics. 3

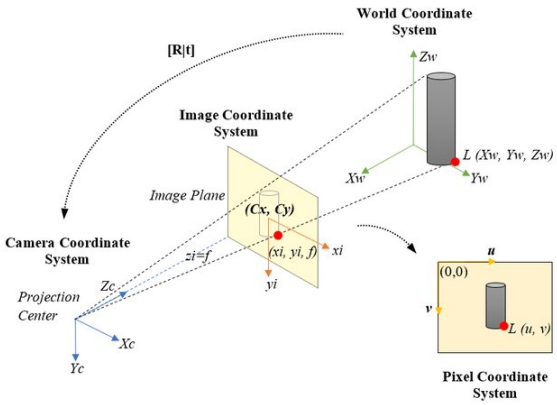


Fig. 3. Simplified Camera model Projection [22]

2) Coordinate Transformation: : The calibration algorithm calculates the camera matrix using the extrinsic and intrinsic parameters. The extrinsic parameters represent the location of the camera in the 3-D scene. The intrinsic parameters represent the optical center and focal length of the camera.

Consequently, in order to project the point in the world frame to the camera image plane, one needs to estimate a

transformation matrix that transforms the world frame into the camera frame.

This transformation is the fundamental idea of camera calibration that one is given a known set of points in the world and their corresponding projections in the image. The main task is to find the camera matrix responsible for the projection transformation.

Camera parameters include intrinsic, extrinsic, and distortion coefficients. To estimate the camera parameters, one needs to have 3-D world points and their corresponding 2-D image points.

3) Extrinsic Parameters: The extrinsic parameters consist of a rotation, R, and a translation, t. The origin of the camera's coordinate system is at its optical center and its x-axis and y-axis define the image plane.

In other words, the transformation from the world coordinate system to the camera coordinate system is called camera extrinsic or camera pose.

So, the camera extrinsic matrix M can be represented in the given form:

$$M = \begin{bmatrix} R_3x3 & t_3x1 \\ 0_1x3 & 1_1x1 \end{bmatrix} \tag{1}$$

4) Intrinsic Parameters: The intrinsic parameters of a camera depend on how it captures the images. Parameters such as focal length, aperture, field-of-view, resolution, etc govern the intrinsic matrix of a camera model. So, generally, The focal length represented in X and Y coordinates f_x,f_y and the camera center represented in X and Y coordinates c_x,c_y are the camera intrinsic parameters, K.

$$K = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \tag{2}$$

In summary, extrinsic parameters give the orientation and location of the camera frame with respect to the real-world frame. On the other hand, intrinsic parameters consist of the internal parameters of the camera and the relationship between camera coordinates and coordinates of the pixels of the digital image in the image frame.

The extrinsic parameters are rotation (3x3) and translation (3x1) matrices, whereas the intrinsic parameters are focal length, skew factor, the optical center of the image, etc. The intrinsic parameters can be shown in a Calibration Matrix, K where f is the focal length and c is the center pixel [12].

B. Convolutional Neural Network

A Convolutional Neural Network (CNN) is a type of deep neural network commonly used in computer vision and image processing tasks. It consists of multiple layers that process and analyzes input images to learn and extract features from them [13].

To detect the AprilTags in a captured image, a CNN structure is selected due to its advantages over other machine learning models such as:

1. CNNs are highly capable of extracting and learning relevant features in the images. In this project, detecting an AprilTag in an image is a perfect task for a CNN, since AprilTags are highly differentiable from other segments of the images [14].

2. CNNs can maintain their robustness even with augmented or distorted images. This feature allows us to use CNNs for not clear images. Moreover, due to their multi-scale processing, they can detect objects in the images even if they appeared in different sizes and orientations. In our project, the AprilTag is being shown to the camera from different angles and sizes, consequently, the comprehensive detection feature of CNNs is crucially useful [15].

3. CNNs can be optimized with end-to-end training. This feature of convolutional neural networks allows us to train them with simple datasets consisting of images and labels [16].

1) Building a Convolutional Neural Network Model for AprilTag detection:: The initial step of training the CNN model was building a suitable data set. To find an existing dataset, extensive research was made, however, there were not any usable datasets. As a result, a brand new dataset is prepared for the training. To prepare the dataset, example images of AprilTags with different sizes and orientations are fetched. Using python image processing libraries, those images are located in different backgrounds, with random sizes and locations. Using this method, more than 2200 unique AprilTag images are created. After that, each image is fed to the existing AprilTag detection function to get their center locations. With the X coordinate of the center locations of images, a label dataset is created. To have a stronger model and overcome overfitting, small augmentations are applied to the dataset without disturbing the AprilTag's center locations which would create a false label. Later the dataset is split to train and validation subsets with the 0.2 ratio.

After the dataset preparation, using python's Keras and TensorFlow libraries, different CNN architectures are built. The main idea behind model architectures was building a small but accurate model. Since the model is prepared for Raspberry Pi, models with fewer layers and neurons are prioritized.

To train the model, the loss function "the mean squared error" is used with the optimizer "Adam", a popular optimization algorithm for training because it is computationally efficient, and has few memory requirements [17]. Also, 32 batch size and the final models are trained for a range of 50 to 100 epochs. To test the models, a new dataset is created with the same methodology mentioned before and 400 new pictures are fed to the model. After the predictions, the error rate is calculated with the absolute distance in the X coordinates between the model predictions and AprilTag detect function. A model with a 10 % error rate and 9 % standard deviation is selected due to its acceptable performance and relatively small size.

IV. Implementation

A. Camera Calibration

Chessboard calibration is a popular method for camera calibration due to being a flat surface, consequently not having the z-axis, and having easily detectable corners and points. Those points appear at straight lines using the Hough transform or many other respectable methods such as Kalman filters.

There are different approaches to camera calibration, the most common ones are Zhang's Method, Tsai's method, and Direct Linear Transformation. Zhang's method uses a checkerboard pattern which needed to be located in at least more than three different positions due to the number of unknowns in the equation.

After extracting the corners of the checkerboard pattern, the method calculates a projective transformation between the captured image points and a scaling factor. Later on, the intrinsic and extrinsic parameters are calculated by solving the equations. On the other hand, Tsai's method assumes that some camera parameters are given, for instance by the camera provider, which significantly reduces the errors in the initial estimation guesses [11], [12], [18].

In this paper, the OpenCV-python library will be used to calibrate the camera. 20 pictures of a 7x9 checkerboard with different positions are collected with miDOG's pi-camera and stored in an external file. Each image is grey scaled and corners of the checkerboards are found using OpenCV's findChessboardCorners function. After, using the calibrateCamera function which utilizes Zhang's method, the calibration matrix for intrinsic parameters and the distortion coefficients are calculated.

1) Distortion in Camera Calibration: Distortion, is a noticeable problem and crucial factor that should be considered during camera calibration.

The lens distortion calibration rectifies the coordinates before they are passed to the spatial calibration. The camera matrix does not account for lens distortion because an ideal pinhole camera does not have a lens. To accurately represent a real camera, the camera model includes the radial and tangential lens distortion.

The physical flaws contained by cameras cause distortions in the images. There exist different forms of distortions. Two significant ones are radial distortion, straight lines appear curved, and tangential distortion, where the actual image appears stretched or tilted in the digital image [10]. The visual examples of the distortion types can be seen in Figure 4.

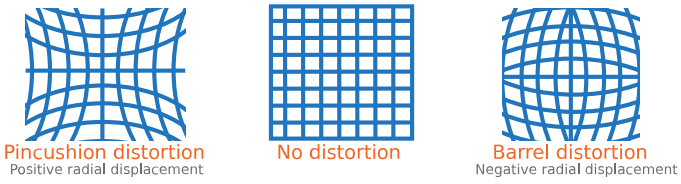


Fig. 4. Different camera distortions [24]

B. Environment setup

Before starting with AprilTag detection process, the environment must be set up. This setup initially includes setting up a global coordinate frame which is the bottom left corner of the room with the X-axis pointing to the right, Y-axis upwards and Z-axis pointing out of the wall. An AprilTag is also placed in its initial position. Furthermore, the position and orientation of the AprilTag with respect to the global coordinate frame are measured to calculate the pose of the camera with respect to the global frame.

C. AprilTag Detection

AprilTag is a visual fiducial system, useful for a wide variety of tasks including augmented reality, robotics, and camera calibration. Targets can be created from an ordinary printer, and the AprilTag detection software computes the precise 3D position, orientation, and identity of the tags relative to the camera. [23]

There are various families of AprilTags, with different advantages and disadvantages. In this paper, the 36h11 tag family is used, as it is one of the most common AprilTags.

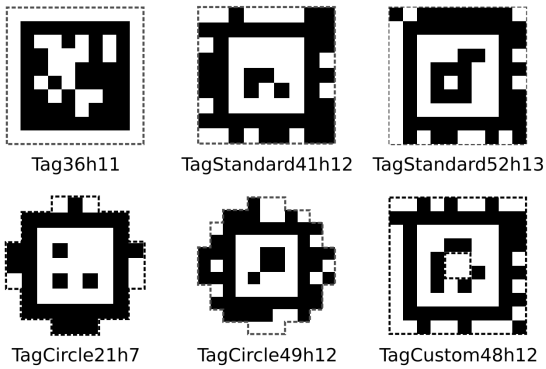


Fig. 5. AprilTags Families [23]

To detect the AprilTag in the captured image, The AprilTag and OpenCV libraries are used. The AprilTag library has a detect function whose algorithm consists of two main methods which are fast corner detection and quadrilateral detection. Fast corner detection algorithms are used to quickly identify potential corners of AprilTags in the image. Later, they are used as starting points for quadrilateral detection. Once potential corners have been identified, quadrilateral detection algorithms search for quadrilaterals which are basically four-sided shapes in the image that match the expected shape of AprilTag markers [20].

Once a marker is detected, the marker's binary payload is decoded to determine the marker's identifier, and the position and orientation of the marker are estimated using methods such as perspective-n-point (PnP) algorithms. Finally, the detected marker is validated using various checks such as verifying the parity of the marker's identifier and verifying that the marker's quadrilateral has the correct shape.

Before calling the AprilTag detect function, the captured images should be grayscaled for better performance. The detect function gives the corners of the AprilTag in the image, which is after used in the OpenCV's solvePnP function with object points of the AprilTag, camera matrix and distortion values from camera calibration to obtain the translation vector. The translation vector consists of the real-word coordinates of the AprilTag.

Firstly, the program reads the Z-axis value to determine the distance between AprilTag and the robot. If the distance is smaller than a given threshold, it stops the robot, since the AprilTag is reached. Otherwise, it continues by reading the X-axis value of the translation vector. Depending on the X-axis value, the program decides either to go straight, turn left or turn right and sends these commands to the robot. Moreover, if the program cannot read an AprilTag ten consecutive times, it sends a command to the Robot to stop and wait for an AprilTag to be detected.

D. Localization and Filtering

Following the AprilTag detection, the pose of the tags in the camera frame is obtained. This information is used to localize the robot with respect to the global frame.

To achieve this, the camera frame should be transformed to the tag frame. By multiplying the rotation matrix of the tag with respect to the camera and the translation vector of the tag with respect to the camera, the exact location of the camera in the tag frame can be obtained. Finally, based on the measurements and angles from the environment setup of the global frame with respect the tags, the tag frame is then transformed to the global frame and thus localizing the robot with respect to the global frame.

Since the readings from the AprilTag are quite noisy with lower accuracy when the distance between the camera and the tag is large,at 1.5 m approximately, filtering is required. The readings are further filtered using a moving average filter at constant time steps to smoothen these readings and improve the measurements.

E. Control System

The main objective of this paper is to track a moving AprilTag which requires some information about the camera model (K matrix, distortion) and the relative position of the camera with respect to the AprilTag local frame. In this section , the complete control system will be illustrated with its main components.

The control process begins with detecting a fiducial marker in front of the camera, followed by the detectors' role to estimate the relative position of the camera and pass it to the controller. Aditionally, the switching controller compares the distance between the camera and the AprilTag, with an initialized safe-distance(0.1m), to ensure that the robot doesn't move beyond this distance. Upon further movement of the tag, that is when the error increases, the motion would then be initiated again.

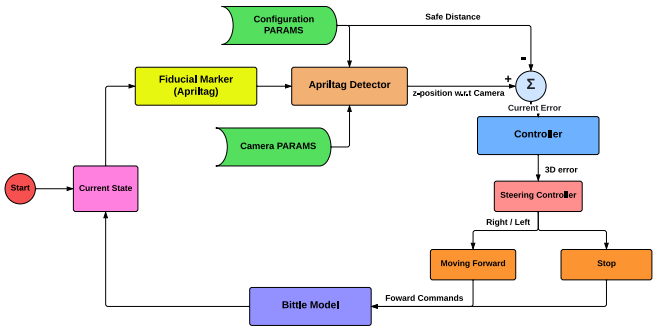


Fig. 6. AprilTag Following Process

Therefore, the main task of the controller is to calculate the current error in 2D, in X and Z directions. An error in the X direction would activate the steering controller to determine the movement direction, and the error in the Z direction would determine the forward movement. Based on these mentioned error results, the servo motors are then actuated.

The steering controller is initially initiated followed by a forward movement controller. The curved motion of the robot might result in a field of view that is unpredictable thus resulting in the loss of detection accuracy, in scenarios of fast-moving AprilTags. Therefore, to avoid this risk, a CNN model is also implemented.

F. Implementation of the AprilTag Detection CNN

The CNN model is built as a backup tool when the AprilTag detect function fails to locate the AprilTag in the image. At the beginning of the script, the model is loaded but only called when the AprilTag detection function returns empty. The CNN model is equipped with a very simplistic approach that can only be used to determine the direction of the robot. By other means, the CNN model cannot predict the position of the AprilTag or the distance between AprilTag and the robot dog. Consequently, the CNN model is only able to tell the robot to turn right / left or go straight. It cannot make a decision for the robot to stop moving. Also, when the CNN model is called, it must return one of those decisions. Even if there are not any April tags in the captured image, it will return a direction. To deal with this issue, a confidence score feature is tried to be implemented in the model. In this feature, if the model's confidence for a prediction is lower than the threshold specified, however, it failed to solve the problem. A cross-validation approach was also tried, yet it failed to give a trustworthy indication of the model's robustness. In the current setting of the project, this issue is left as future work.

V. Results

The results of testing in a real-world environment of the Quadruped robot equipped with a camera for fiducial marker detection will be illustrated in this section. A simulation environment is also illustrated in this section.

A. Simulation Results

By computing the inverse kinematics during motion, which was not possible during the experimental implementation because of various hardware limitations, the AprilTag detection and navigation process missions were tested using Gazebo virtual world and Robot Operating System (ROS), for additional illustration. ROS was chosen to arrange the task architecture rapidly because of the advantages of its simpler hardware abstraction and code reuse, and due to Gazebo's interoperability with ROS, it was chosen as the primary platform to mimic the designated motion of a virtual motion.

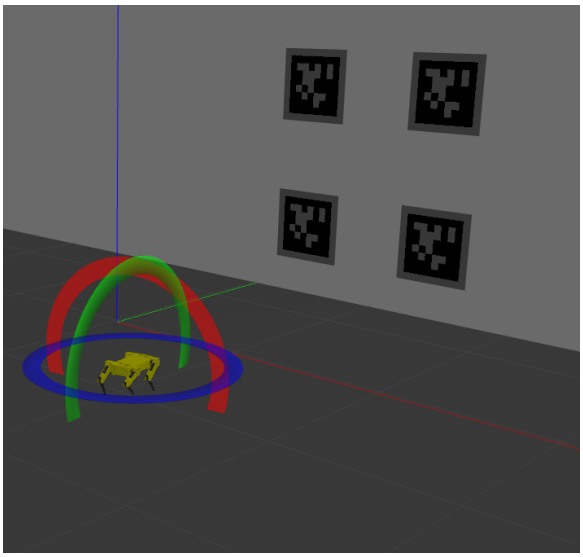


Fig. 7. Simulated AprilTag Following Process on Gazebo

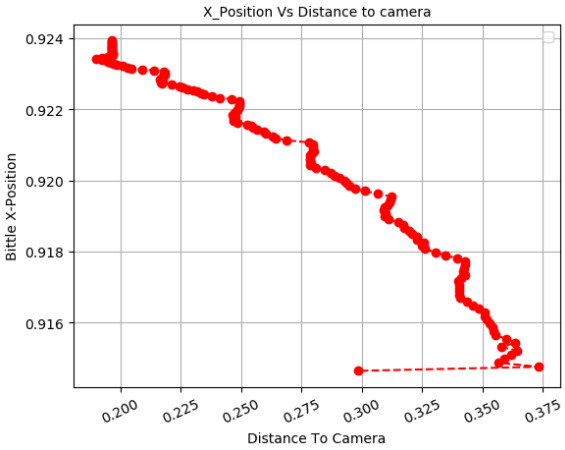


Fig. 8. Simulated Apriltag tracking data

As seen in 8, the decay in the robot X-position was purely linear due to the robot dynamics and the body inertial which has not been precisely calculated in the simulation, but overall, the detection process of AprilTag was fairly good and the robot managed to detect and track it.

B. Experimental Results

1) AprilTag Follower Results: Initially, only the detection of the AprilTag was tested in a controlled environment with two lighting setups. The detection algorithm achieved a success rate of 95% in accurately detecting the marker, where a successful and failed detection is demonstrated in figures 8 and 9. A faster movement of the marker would result in a lower success rate due to blurriness and camera quality. Furthermore, with dim lighting a tag detection range of 0.8m was achieved, while a range of 1.5m was achieved with brighter lighting. Additionally, an error of 0.15 m between the real position and the estimated position realized from detecting the AprilTag was recorded.

Since the readings received from tag readings are quite noisy, the data were smoothened prior to sending it to the controller. In Figure 8, the result of applying a moving average filter is depicted.

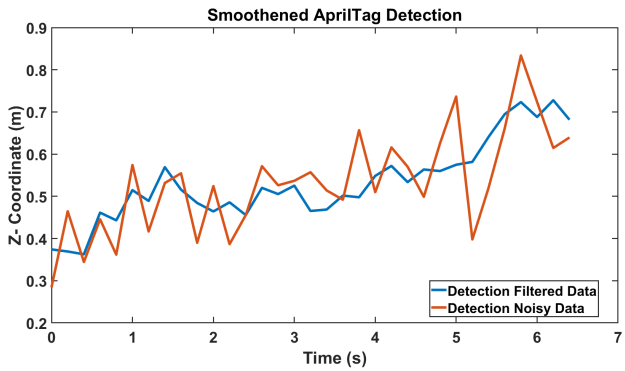


Fig. 9. Smoothed AprilTag Detection

Following a dynamic moving tag using the previously explained controller was also successful. As seen in Figure 9, tracking the tag was accurate with the safe distance between the robot and the tag maintained.

2) AprilTag Follower Results with CNN Support: After the tryouts with two different scripts, one without and one with CNN support, it has been revealed that CNN support helps the robot to move continuously, yet not necessarily in the correct direction all the time. However, since the images are captured at a high rate, the CNN network made the robot continue moving in the correct direction until the AprilTag detection function detects the AprilTag. Moreover, there was a small performance reduction in the detection speed. In figures 8 and 9 example results can be seen. In Figure 8, the detect function can successfully detect the AprilTag in the image and return to the correct command of turning right. However, in Figure 9, the detect function cannot detect the AprilTag. In this situation, the CNN model was able to give the correct decision.

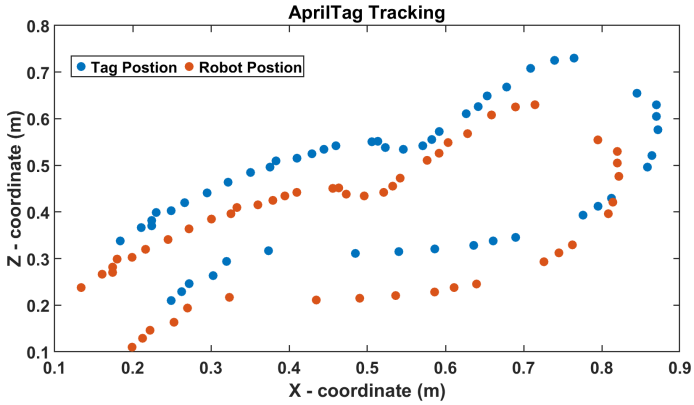


Fig. 10. AprilTag Tracking Data

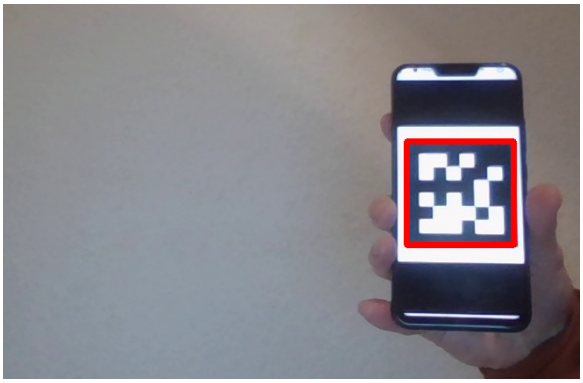


Fig. 11. Successful AprilTag Detection by the detect function

It is decided that having a reliable CNN model can significantly improve the outcome of the project and the following performance of the robot. Nevertheless, to achieve a fast and highly accurate convolutional neural network, a significant amount of computational power and a more natural and varied dataset might be needed for both training and predicting.

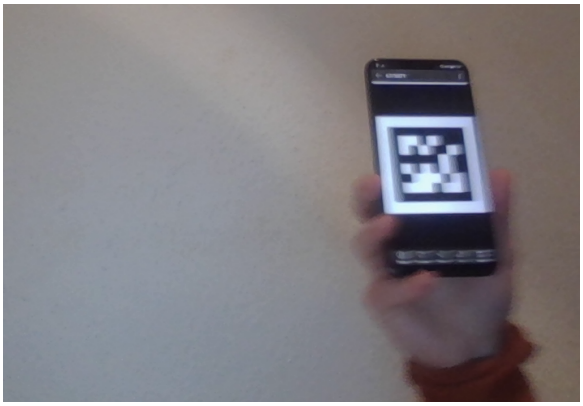


Fig. 12. Unsuccessful AprilTag Detection by the detect function

VI. Discussion and Conclusion

In conclusion, this paper presented a robust method for fiducial marker detection and following using quadruped robots. The results showed that the proposed method was effective and robust in achieving stable marker tracking in real-world environments. Further improvements in AprilTag detection is required, by including a Kalman filter for example instead of the moving average filter, to improve the accuracy of these detections. Furthermore, a higher computational power and a more varied data set would also aid in improving the CNN model. This study highlights the potential of the proposed method to enable new applications for quadruped robots in hazardous and challenging environments. Further research is recommended to continue exploring the capabilities and limitations of the proposed method.

Furthermore, a higher computational power and a more varied data set would also aid in improving the CNN model. This study highlights the potential of the proposed method to enable new applications for quadruped robots in hazardous and challenging environments.

References

[1] Yu Zhang, et al. Frontiers in Robotics and AI, 2019. Robot-Oriented Design. A Review of Autonomous Robots for Precision Agriculture.
[2] Rong Xiong, et al. International Journal of Robotics and Automation, 2018.Path Planning for Autonomous Environmental Monitoring Robots.
[3] Xiaoguang Liu, et al. Robotics and Computer-Integrated Manufacturing, 2014.Path Planning for Autonomous Mobile Robots in Manufacturing Environments.
[4] Edwin Olson, et al. International Journal of Robotics Research, 2010.Robust and Efficient Fiducial Marker Detection for Mobile Robots.
[5] Edwin Olson, et al. International Conference on Robotics and Automation, 2009.Scalable Fiducial Markers for Mobile Robots.
[6] Wei He, et al. International Conference on Robotics and Automation, 2012.Fast and Accurate AprilTag Detection on a Mobile Phone.
[7] Wei He, et al. International Conference on Robotics and Automation, 2013.AprilTag Detection in Low-Light Conditions.
[8] Wei He, et al. International Conference on Robotics and Automation, 2014.Range Extension for AprilTag Detection.
[9] Baláž, V., Vagaš, M., Semjon, J., Rusnák, R. Camera Systems of Industrial Robots. Range Extension for AprilTag Detection
[10] Szeliski R. Computational Photography Computer vision: algorithms and applications. London: Springer. 2011
[11] Alqahtani, Faleh - Banks, Jasmine, Chandran, V., Zhang, Jinglan. Detection and Tracking of Faces in 3D Using a Stereo Camera Arrangements. International Journal of Machine Learning and Computing. 2019
[12] Zhang, Zhengyou. A Flexible New Technique for Camera Calibration. Pattern Analysis and Machine Intelligence, IEEE Transactions.2000
[13] M. Elgendy, "Deep Learning for Computer Vision: A Beginner's Guide," Packt Publishing Ltd, 2020.
[14] R. Szeliski, "Computer Vision: Algorithms and Applications," Springer Science Business Media, 2010.
[15] R. Shanmugamani and A. Ray, "Deep Learning for Computer Vision," Apress, 2020.
[16] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770-778.
[17] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770-778.
[18] Roger Y. Tsai. A Versatile Camera Calibration for High-Accuracy 3D Machine Vision Metrology Using Off-the-Shelf TV Cameras and Lenses, IEEE Journal of Robotics and Automation, Vol. RA-3, No.4. 1987
[19] Remondino, Fabio, Fraser, Clive. Digital camera calibration methods: Considerations and comparisons. Ine. Arch. Photogramm. Remote Sens. Spat. Inf. Sci.. 36. 2015

[20] Edwin Olson. AprilTag: A robust and flexible visual fiducial system, IEEE International Conference on Robotics and Automation (ICRA), 2011
[21] Zhang, Z. A flexible new technique for camera calibration. *IEEE Transactions On Pattern Analysis And Machine Intelligence*. **22**, 1330-1334 (2000)
[22] Fernandez, L., Avila, V. & Gonçalves, L. A generic approach for error estimation of depth data from (stereo and RGB-D) 3D sensors. (Preprints,2017)
[23] Haggenmiller, A. Non-parametric Models for Long-term Autonomy. (University of Michigan,2022,7)
[24] Heikkila, J. & Silven, O. A four-step camera calibration procedure with implicit image correction. *Proceedings Of IEEE Computer Society Conference On Computer Vision And Pattern Recognition*. pp. 1106-1112 (1997)
[25] Wang, J. & Olson, E. AprilTag 2: Efficient and robust fiducial detection. *2016 IEEE/RSJ International Conference On Intelligent Robots And Systems (IROS)*. pp. 4193-4198 (2016)
[26] Olson, E. AprilTag: A robust and flexible visual fiducial system. *2011 IEEE International Conference On Robotics And Automation*. pp. 3400-3407 (2011)